

Vrije Universiteit Amsterdam



Bachelor Thesis

Implementing a video recommendation system for educational platforms

Author: Matei Ștefan Avram (2725281)

1st supervisor: Thilo Kielmann
2nd reader: Felienne Hermans

*A thesis submitted in fulfillment of the requirements for
the VU Bachelor of Science degree in Computer Science*

July 4, 2024

Abstract

This thesis focuses on the application of recommendation systems within the context of online educational platforms. Popular services like Khan Academy do not attempt to keep students engaged by providing them with content they are interested in, nor do they offer any guidance when it comes to students orienting themselves for further education. A cosine similarity based recommendation algorithm that aims to profile interests in various disciplines, could prove useful, in aiding with the orientation process as well as keeping students engaged. Thus, the question “To what extent can the cosine similarity based recommendation algorithm implemented (in this thesis) can recognize the interests of; and recommend educational content to users?” is explored. An experiment was conducted using a simulated environment consisting of 100 bots and 5000 pseudo-generated videos to determine the optimal minimum similarity threshold for the given implementation. After results were compiled, the optimal value was observed to be 0.4 and it was concluded that the recommendation algorithm provided, proved effective in profiling user preferences and recommending relevant content. The approach could be incorporated into educational platforms, to help students better clarify their list of disciplines they are interested in, thus aiding in the orientation process for further education.

Contents

1	Introduction	4
2	Related Work	5
3	Research Method	6
3.1	Recommendation System Implementation	6
3.2	Platform Implementation	9
3.3	Experimental Setup	10
4	Results	14
4.1	Example Run	14
4.2	Summary	17
5	Discussion	18
5.1	Observations	18
5.2	Limitations	19
6	Conclusion and Further Exploration	20
	List of figures	21
	References	22

Chapter 1

Introduction

As technology continues to grow at a rapid pace, driven by computational and algorithmic advancements, the amount of information generated has increased as well. To efficiently access relevant information, recommendation systems were developed (Kreutz & Schenkel, 2022). Recommendation algorithms are a form of information filtering systems that attempt to provide a user with content tailored to its interests (Roy & Dutta, 2022).

When it comes to the field of social media, large corporations that control platforms like TikTok, YouTube and Instagram have dedicated a lot of time and effort in researching recommendation systems, in an effort to keep users engaged for longer periods of time. Similarly, the educational field could also benefit from the introduction of a recommendation system in online educational platforms as such services could take a more direct approach in guiding individuals to lead successful and fulfilling lives, besides providing high quality educational content. Currently, one of the most popular platforms called Khan Academy focuses on providing a complementary resource to classroom learning, offering a library of online courses across many subjects. (Khan Academy, 2024a). While it makes an effort in structuring material in a digestible way for students, the platform does little to aid students in the process of orientation for further education.

A recommendation algorithm could prove to be a useful addition to educational platforms as the profiling capability of the algorithm might help identify students' interest in different disciplines. By communicating these interests to individuals who have trouble identifying them, it could aid them in the orientation process. Moreover, by recommending content of interest to students, it could have a similar effect observed in social media applications where user engagement is increased (Wang, 2022).

Considering the potential benefits, an attempt was made to create a personal implementation of a cosine similarity based recommendation algorithm, that is tailored for the specific use of profiling students' interest in different disciplines accurately. Thus, this thesis attempts to answer the research question "To what extent can the cosine similarity based recommendation algorithm implemented can recognize the interests of; and recommend educational content to users?".

To answer the proposed question, the algorithm will be evaluated with a simulated environment consisting of 100 bots and 5000 pseudo-generated videos. The bots are meant to simulate students utilizing the platform. For this implementation five runs will be conducted that aim to determine the optimal minimum similarity threshold study parameter. Additionally, an educational platform demo will be created to showcase how such an algorithm could be incorporated. Although user engagement might be a potential benefit it is not measured in this thesis. The findings of this study could provide valuable insights into the application of recommendation systems in educational platforms, and how these can expand their current functionalities.

Chapter 2

Related Work

To create a recommendation system, two fundamental components must be implemented. The first is item representation which refers to how the items to be recommended and the users being profiled are represented. This is primarily achieved through vector embeddings. The second component is the filtering method chosen. There are two primary filtering methods available, which are Content-based filtering (CBF) and User-based filtering (UBF) (Roy & Dutta, 2022).

A user's interests and the content they consume need to be represented in such a way that computer algorithms and mathematical functions can be applied, to produce a result on whether an item should or should not be recommended. Borrowing from the field of natural language processing research, a method used to accomplish this is through vector embeddings (Rizkallah et al., 2021). These are "a method of converting words, sentences and other types of data or information into numbers that capture their meanings and relationships" (Elastic, 2024). While there are multiple types of embeddings such as document embeddings and image embeddings, the two types used in this thesis are video and user vector embeddings. The difference between these different types comes from the implementation or the algorithm used to generate these vectors.

Similar to video sharing platforms like TikTok and YouTube, the items being recommended on the demo educational platform implemented in this thesis are videos. These have a multitude of features such as: title, tags (pertaining to topics discussed in the video) and a list of categories that the video can belong to. A vector which represents the video could be generated from multiple features such as the ones previously described (*Recommender Systems Handbook*, 2022). Similarly, a user could also be represented by the vector generated from the same features enumerated, of multiple videos they watched.

For this implementation, to identify the disciplines a user is interested in, which can then be used to assist in the orientation process, videos will be represented as vectors with each dimension corresponding to a discipline available in the application. Similarly, users will be represented by vectors where each dimension indicates the user's interest level in the respective discipline.

Following the representation of users and content, filtering methods are applied such that the recommended content aligns with the user's interests. As stated before there are two main filtering methods CBF and UBF. Content-based filtering recommends items to users based on other material they have positively engaged with, whereas user-based filtering recommends items by considering what material has been positively received by other users with similar preferences (Roy & Dutta, 2022).

A potential drawback of CBF, is that it demands an in-depth knowledge of an item's features i.e titles, tags and categories which are needed for accurate recommendations, yet these are not always available. Alternatively, it does have the ability to quickly adapt to changes in user preferences. The main disadvantage of UBF is the "cold start problem". When a new account is created, depending on the implementation, the algorithm does not have any insight into what

the user preferences are, and thus fails to identify “neighboring” users accurately. Nevertheless, it does not require any features of the content being recommended i.e. videos. At the cost of increased algorithm complexity, there is also the option of Hybrid-based filtering (HBF), which is a combination of the two methods previously described that is said to increase recommendation performance (Roy & Dutta, 2022).

For the implementation of the recommendation system in this thesis, content based filtering was used. This choice was made because the algorithm was evaluated in a simulated environment with a pseudo-generated video dataset, ensuring that item features i.e. the discipline each video belongs to, are always available, thus addressing the primary limitation of CBF. Moreover, algorithm complexity is kept at a minimum and this approach eliminates the cold start problem.

Finally, it is important to apply the right algorithms for the chosen filtering method. A possible configuration for a HBF recommendation system was described in (Singh et al., 2020). Cosine similarity is used to ensure the functionality of the CBF method while a K-Nearest Neighbour (KNN) machine learning model is implemented to group users accurately for the UBF method. Cosine similarity is a mathematical formula that can be utilized to compare two vectors with the same number of dimensions and derive a conclusion as to how similar they are. This makes it a good fit for recommendation systems that utilize vector embeddings to represent items, like the one implemented in this thesis. Additionally, there are some potential performance benefits that will be discussed later. As for KNN, it is a solid choice for grouping similar user vectors as the algorithm is specifically tailored for classification (Cunningham & Delany, 2021).

Chapter 3

Research Method

This section outlines the steps taken to implement the cosine similarity based recommendation algorithm, the demo educational platform and the experimental setup. The methodology is structured as follows: first, the representation of data and the implementation of the recommendation system are described. Next, the implementation of the platform is detailed, followed by the experimental setup used to evaluate the system. Finally, a summary of the methodology is provided.

3.1 Recommendation System Implementation

To set a solid foundation for building the algorithm, it is important to establish how data will be represented and how videos will be filtered. As stated prior, users and videos were both represented by vectors of 10 dimensions where each dimension is equivalent to the following disciplines in order: Arts, Biology, Business, Chemistry, Computer Science, Literature, Mathematics, Music, Physics, Psychology. These have been selected as they offer a balanced set of both humanitarian and scientific fields of education. The mathematical representations for the user and video vectors are:

$$\mathbf{u} = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{10} \end{pmatrix} \quad \mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_{10} \end{pmatrix}$$

To ensure a consistent scale the user vector has to be normalized. Vector normalization is a process by which you take a vector of any length and change it such that it maintains the same direction but it is now of magnitude 1, otherwise known as the unit vector (Khan Academy, 2024b). This is achieved by employing the use of the following formula:

$$\frac{\mathbf{v}}{\|\mathbf{v}\|} = \begin{bmatrix} \frac{v_1}{\|\mathbf{v}\|} & \frac{v_2}{\|\mathbf{v}\|} & \cdots & \frac{v_n}{\|\mathbf{v}\|} \end{bmatrix} \quad (1)$$

where

$$\|\mathbf{v}\| = \sqrt{\sum_{i=1}^n v_i^2} = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

is the Euclidean Norm provided by `numpy.linalg.norm` method (NumPy Developers, 2024). The mathematical formula is described in (*Data Mining: Concepts and Techniques*, 2012).

The cosine similarity function was used to achieve the functionality of the content-based filtering method chosen. It is able to measure how similar two vectors are i.e the video and user vector, by evaluating if they are pointed in roughly the same direction, using the cosine of the angle between them (*Data Mining: Concepts and Techniques*, 2012). The formula used to compute this similarity is the following:

$$S(\mathbf{v}, \mathbf{u}) = \frac{\mathbf{v} \cdot \mathbf{u}}{\|\mathbf{v}\| \|\mathbf{u}\|} \quad (2)$$

where $\mathbf{v}$ and $\mathbf{u}$ are the video and user vectors respectively, and

$$\mathbf{v} \cdot \mathbf{u} = \sum_{i=1}^n v_i u_i = v_1 u_1 + v_2 u_2 + \cdots + v_n u_n$$

is the Dot Product of 2 n-dimensional vectors provided by `numpy.dot` method (NumPy Developers, 2024). The output scale of the cosine similarity function ranges from $[-1, 1]$ where -1 indicates negative similarity (vectors are pointed in opposite directions) and 1 indicates perfect positive similarity. A computational efficiency can be achieved if the vectors used when computing cosine similarity are stored in normalized form. Because their magnitude now equals to 1, function (2) now transforms into:

$$S(\mathbf{v}, \mathbf{u}) = \mathbf{v} \cdot \mathbf{u}$$

Two main methods need to be implemented for the recommendation system. First, the user has to be able to request videos to watch, then it should be able to send feedback to the platform.

When a user makes a request, videos are retrieved from the database and 10 are selected to be sent to the user. To ensure the algorithm is able to profile the interest in disciplines accurately, the first 500 most recent videos are sent to the user regardless of their interest. After this requirement is met (some preferences are recorded), the user starts receiving videos based on their interests. To ensure the most relevant videos are sent first, the algorithm starts with an initial threshold of 0.90 for the cosine similarity function and gradually decreases to the minimum similarity threshold (study parameter) chosen. The pseudocode version of the algorithm can be observed in **Fig. 1**.

Video Retrieval

```
videos ← NewList()
threshold ← 0.9
offset ← 0
retrievedVideos ← database.getVideos(index = offset, limit = 50, orderBy = 'publishDate')
while size(videos) < 10 and retrievedVideos ≠ [] do
  for each video in retrievedVideos do
    similarity ← cosineSimilarity(userVector, videoVector)
    if similarity ≥ threshold and size(videos) < 10 then
      AddItem(videos, video)
    end if
  end for
  if size(videos) == 10 then
    continue
  end if
  if threshold > minimumThreshold (study parameter) then
    threshold ← threshold - 1
    continue
  end if
  threshold ← 0.9
  offset ← offset + 50
  retrievedVideos ← database.getVideos(index = offset, limit = 50, orderBy = 'publish-
Date')
end while
return videos
```

Figure 1: Video Retrieval pseudocode

After each video is seen, a user’s client sends data back to the platform to record the actions taken. A recorded action includes the following details: the user who performed the action, the video on which the action was taken, the duration of the action indicating the watch time, the type of action (such as liking or disliking the video), and a timestamp indicating when the action occurred. After each recorded instance, with the user vector and the given rates vector (that describes the rate at which each category should increase):

$$\mathbf{r} = \begin{pmatrix} r_1 \\ r_2 \\ \vdots \\ r_{10} \end{pmatrix}$$

the following function is applied to update the user vector for each element, depending on the action type:

$$f(u_i) = u_i + ((\text{like rate OR dislike rate}) + \text{watch rate}) * r_i$$

The rates chosen can be observed in **Table 1**. The watch rate is set to -0.01 for a watch percentage below 25%, as this reflects users’ lack of interest in the content, leading them to skip the video. Finally, the user vector is normalized by employing the use of **(1)**.

	Condition			
	None	watch percentage < 0.75	watch percentage < 0.50	watch percentage < 0.25
Like rate	0.1	x	x	x
Dislike rate	-0.05	x	x	x
Watch rate	0.06	0.04	0.02	-0.01
Category rates vector	0.1 for all disciplines	x	x	x

Table 1: Algorithm Update Rates

3.2 Platform Implementation

The chosen configuration for the educational platform is a web-based application with a client and server side. The sole purpose of this demo is to showcase how a recommendation algorithm coupled with short-style video content borrowed from platforms like TikTok and YouTube can be introduced in an educational setting. No benefits are studied.

For the client side implementation, a code stack configuration of HTML, CSS, JavaScript and Bootstrap Framework was used to create a minimalistic and responsive Graphical User Interface. Three main pages are available: Login, Sign Up and finally For You Page where users watch recommended content. These are illustrated below in **Figs. 2, 3, 4**.

For the server side implementation the Python programming language was used as it is flexible with its large number of accessible libraries both for network operations and scientific purposes. This allowed the recommendation algorithm which uses primarily scientific libraries to connect directly with the server rather than trying to run scripts by executing a python file from another programming language. The API was built with the Flask library to easily set up a development server. The main methods available are: /signup, /login, /request_videos and /user_action, which as the names suggest, allow a user to create an account on the platform, log in, request videos to watch and send data analytics to the server respectively.

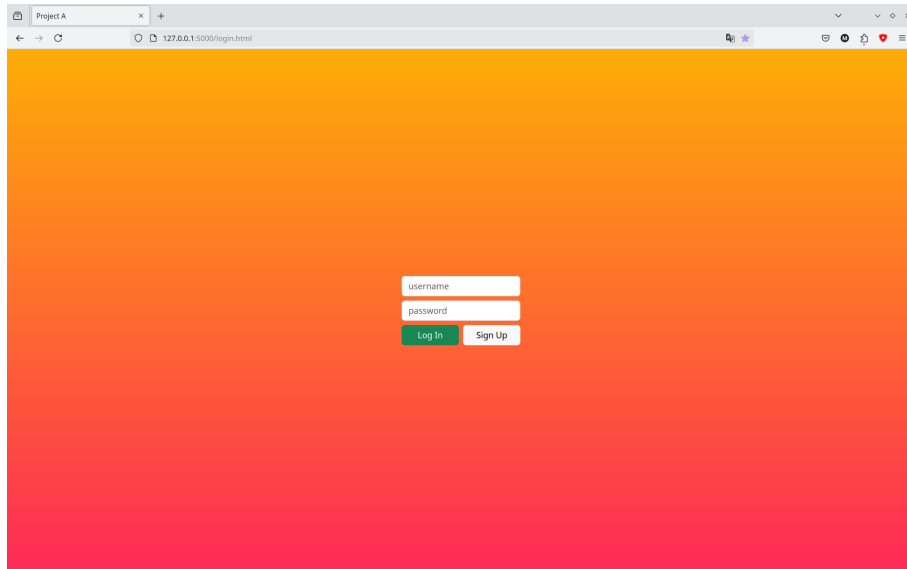


Figure 2: Login Page Interface

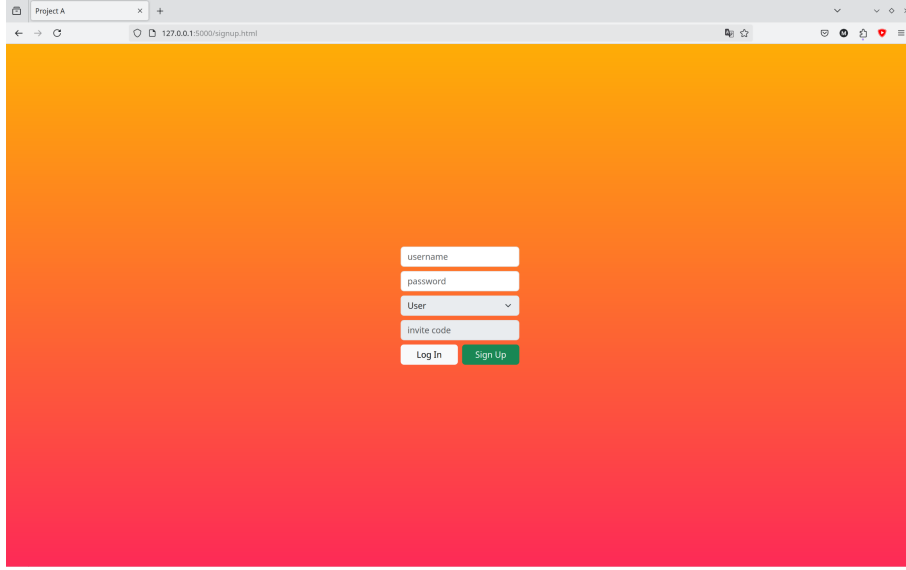


Figure 3: Sign Up Page Interface

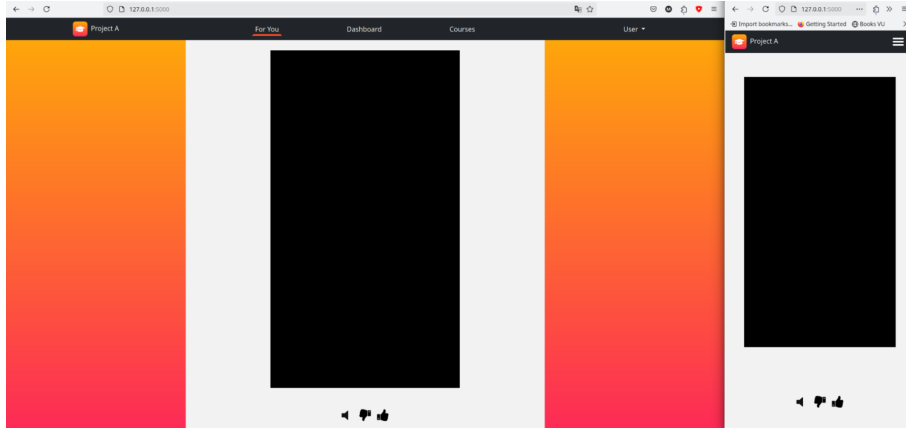


Figure 4: For You Page Interface

3.3 Experimental Setup

To evaluate the recommendation system implemented, a simulated environment of 100 bots was created. These, requested videos, watched them and finally sent feedback to the algorithm through the demo platform implemented. To assess the profiling capabilities, the same cosine similarity function (2) was used to calculate the similarity score between the bot vectors generated by the simulated environment and the user vectors. Recommendation capability performance was estimated by looking at the distributions of watch percentages per discipline for each bot profile. Five runs were conducted to determine the optimal minimum similarity threshold for the provided implementation with values ranging from 0.3 to 0.7. Since the bots were generated with different interest levels for each discipline, an attempt was made to observe whether the recommendation system created was able to detect these.

When creating a simulated environment, it should closely replicate real life to accurately mimic deployment conditions. Thus, the bot profiles were created based on an enrollment by field dataset (OECD, 2023). Data was formatted to extract the total number of people enrolled in each field. Following this, each field was attributed to a combination of one or multiple original disciplines. For the enrollment fields that were not able to be described by a combination

of disciplines or were ambiguous, the count was summed up into a separate class. From the extracted and matched data a distribution was created. The available profiles and the chances of getting each profile are illustrated in **Table 2**.

Profile	Disciplines of interest	Percentages
agriculture, forestry, fisheries and veterinary	bio, chem	2%
arts and humanities	arts, lit, music, psy	8%
business, administration and law	bus, lit	20%
engineering, manufacturing and construction	math, phy	13%
health and welfare	bio, chem, psy	12%
information and communication technologies	cs, phy	4%
natural sciences, mathematics and statistics	bio, chem, math, phy	4%
science, technology, engineering and mathematics	cs, math, phy	20%
services	bus, psy	4%
social sciences, journalism and information	lit, psy	6%
other		7%

Table 2: Bot Profiles distribution

Moreover, to mimic how individuals can have varying interest levels, a random score ranging from 2-5 was assigned to disciplines comprising the profile selected. The interest level metric is the following: 1 - low interest, 2 - slight interest, 3 - moderate interest, 4 - high interest, 5 - very high interest. In order to prevent overfitting, distributions of watch percentages of videos were created using beta distributions through the *scipy.stats.beta.rvs* method, which utilizes the following probability density function to generate the distribution (SciPy Developers, 2024):

$$f(x, a, b) = \frac{\Gamma(a + b) x^{a-1} (1 - x)^{b-1}}{\Gamma(a) \Gamma(b)}$$

where

$$\Gamma(z) = \int_0^\infty t^{z-1} e^{-t} dt$$

and a (α) and b (β) are shape parameters. α is the first shape parameter which primarily influences the distribution's skewness. A negative skewed distribution otherwise known as left skewed, has its left tail longer (skewed) such that the majority of data points are concentrated at higher values, whereas a positive skewed distribution (right skew), is the exact opposite (OpenStax, 2024). When $\alpha < \beta$ the distribution is right skewed whereas when $\alpha > \beta$ the distribution is left skewed. To obtain a roughly normal shaped distribution, α needs to be equal to β . The second shape parameter works in conjunction with α to determine the overall shape of the distribution. The shape parameters were calculated mathematically to attempt the generation of distributions resembling real life data. The interest level distributions are visualized in **Fig. 5**.

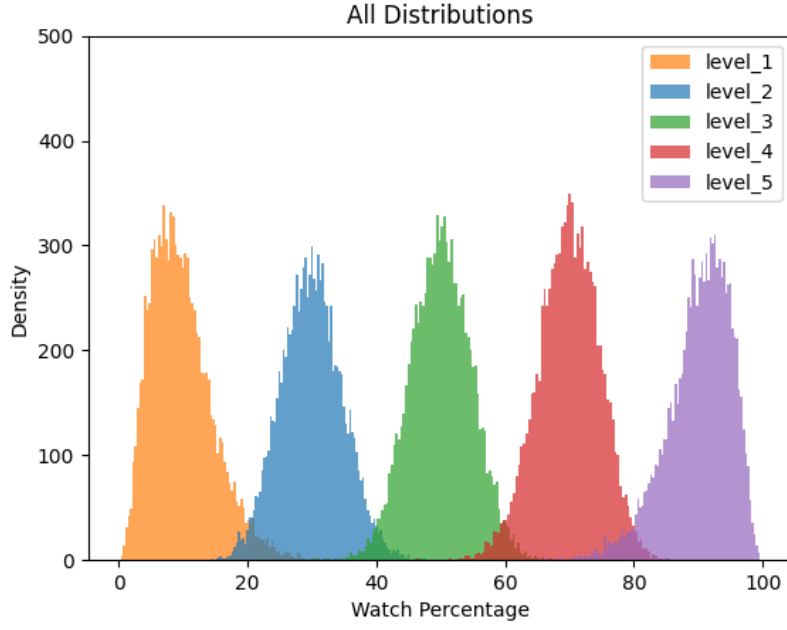


Figure 5: Interest Level distributions

After the profiles and interest levels are assigned, each bot requests videos and sends feedback to the platform about each video seen. Once the platform has no more videos to recommend, the algorithm moves on to the next bot. This was done to better assess the profiling capabilities of the recommendation system implemented, however, it comes at the expense of video recommendation evaluation data. The pseudocode of the algorithm briefly described can be observed in **Fig. 6**.

Simulated Environment Algorithm

```
interestLevelDistributions ← generateInterestLevelDistributions()
bots ← NewList()
for  $i \leftarrow 0$  to nrBots do
  disciplines ← random(profileDistribution)
  if disciplines == " " then
    continue
  end if
  levels ← [random.number(2, 5) for discipline in disciplines]
  bot ← Bot(disciplines, levels)
  bot.generateVector()
  database.insertBot(bot)
  AddItem(bots, bot)
end for
for each bot in bots do
  bot.createAccount()
  bot.logIn()
  videoList ← bot.requestVideos()
  while videoList != [] do
    for each video in videoList do
      videoDiscipline ← video.getDiscipline()
      interestLevel ← bot.getInterestLevel(videoDiscipline)
      watchPercentage ← random(interestLevelDistributions.get(interestLevel))
      actionType ← chooseAction(watchPercentage)
      watchTime ← watchPercentage * video.duration // 100
      data ← {
        'videoId': video.id,
        'actionType': actionType,
        'duration': watchTime }
      bot.sendData(data)
    end for
    videoList ← bot.requestVideos()
  end while
end for
```

Figure 6: Simulated Environment pseudocode

Finally, to attempt the evaluation of whether interest levels were recognized, Spearman's rank correlation was used. This method automatically converts the vectors in ranks and then it measures how strong the relationship between the rankings of the two vectors is (GeeksforGeeks, 2024). Similar to the cosine similarity function, the range of the output is $[-1, 1]$ where -1 indicates perfect negative correlation, 0 indicates no correlation and 1 indicates perfect positive correlation.

In summary, 100 bots were created based on real-life data. Each bot requests videos from the platform until there is nothing left to recommend. On each video request, the platform retrieves 50 initial videos from the database. It loops through the videos and computes the cosine similarity between the video and user vectors. If the similarity score is higher than the current threshold, the video is stored in a list. If 10 videos are stored, they are recommended to the bot; otherwise, the similarity threshold is lowered until it reaches the minimum similarity threshold, representing the study parameter for this implementation. If 10 videos have not been

stored, another 50 are retrieved from the database, and the process repeats. After the videos are recommended, a bot loops through all the videos received. The bot identifies the corresponding interest level for the discipline the video belongs to. Based on the distribution of the interest level, the bot retrieves a random watch percentage. After some computations, the bot sends feedback to the platform regarding the video it just watched. Five runs were conducted to determine the optimal study parameter. To evaluate the profiling capabilities of the algorithm, we studied the distribution of the cosine similarity values, in addition to the distribution of the Spearman’s rank correlation values between the bot and user vectors. Finally, recommendation performance was evaluated by studying the watch percentages of disciplines for each unique profile.

Chapter 4

Results

In the runs conducted, the generated dataset consisted of 5000 pseudo-videos. To showcase how the study parameter affects vector similarity scores between bots and users, a similarity distribution was computed for each run. A histogram was utilized to illustrate and gain significant insights into the similarity distributions with bins representing intervals of 0.05 starting from -0.1 to 1. The similarity score is displayed on the x-axis while the number of scores in that bin are represented on the y-axis. An example similarity score of 0.91 would fall in the [0.9 - 0.95] range, which equates to the 0.9 bin in the histogram. To showcase how well videos were recommended to users for each study parameter value, this thesis employs the use of pie charts to illustrate the largest watch percentages of each unique discipline in relation to their matching profile population.

4.1 Example Run

The following graphs have been compiled for run 2 of the algorithm where the minimum similarity threshold selected was 0.40. In **Fig. 7** the similarity scores of bots and the user vectors that were generated by the algorithm seem to define a highly left skewed distribution with the majority of the similarity scores falling between 0.9 and 1.0 and the mode residing at 0.95 holding 73 out of 94 data points. **Fig. 8** showcases the view counts of all the bots in run 2. The illustration depicts a slightly right skewed distribution with its peak at around 1400 to 1500 videos watched. The average view count of bots in this experiment is 1647. Majority of pie charts in **Fig. 9** depict a matching discipline distribution according to the profile described above. Not all discipline percentages, that make up the profile, are equally represented with the ‘others’ section, exceeding the value of 10% in 2 out of 10 charts.

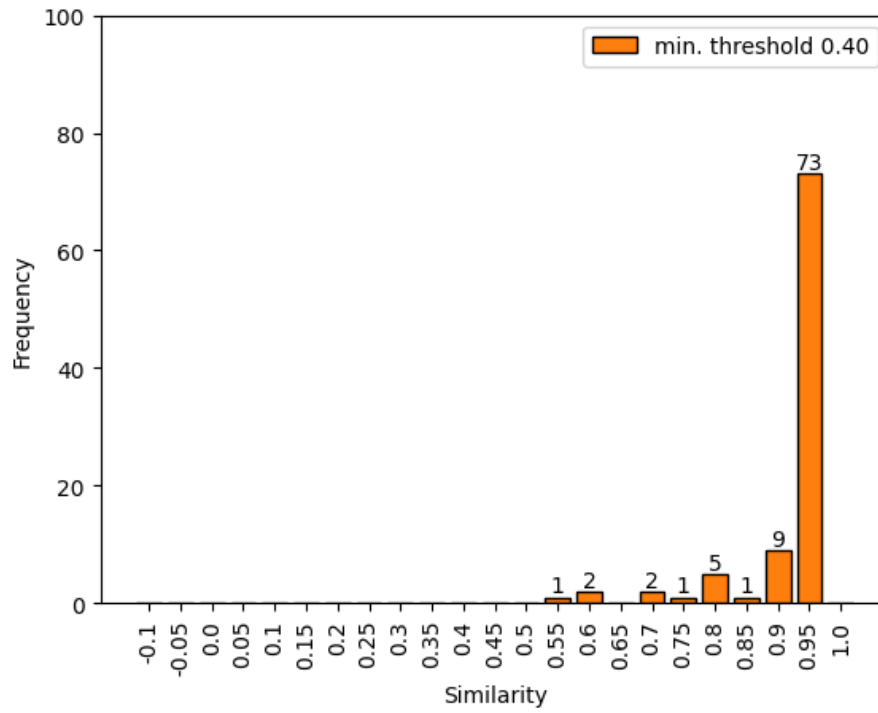


Figure 7: Bot and User Vector cosine similarities distribution

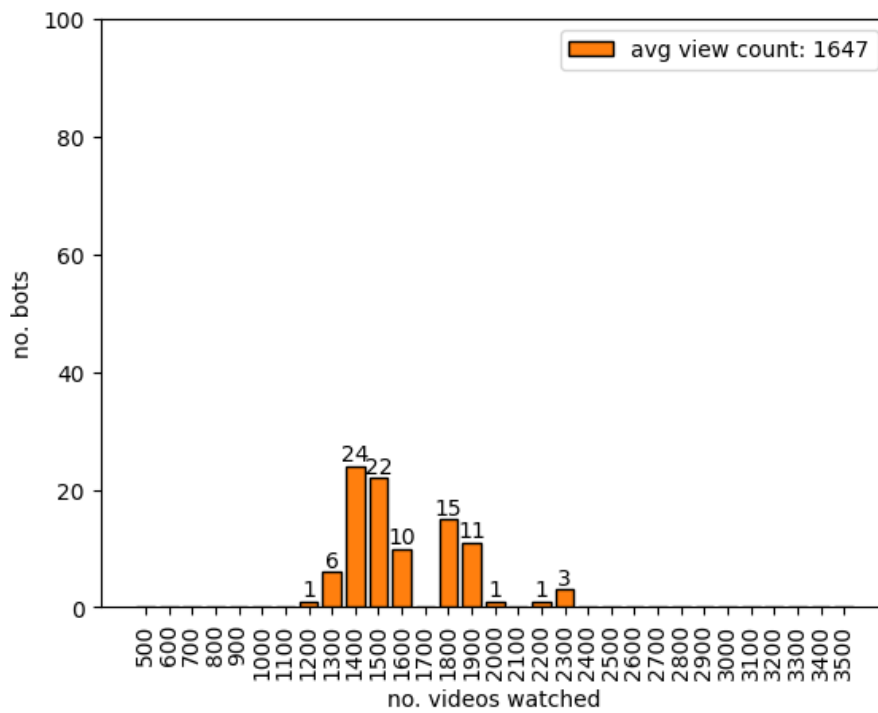


Figure 8: Bot view counts

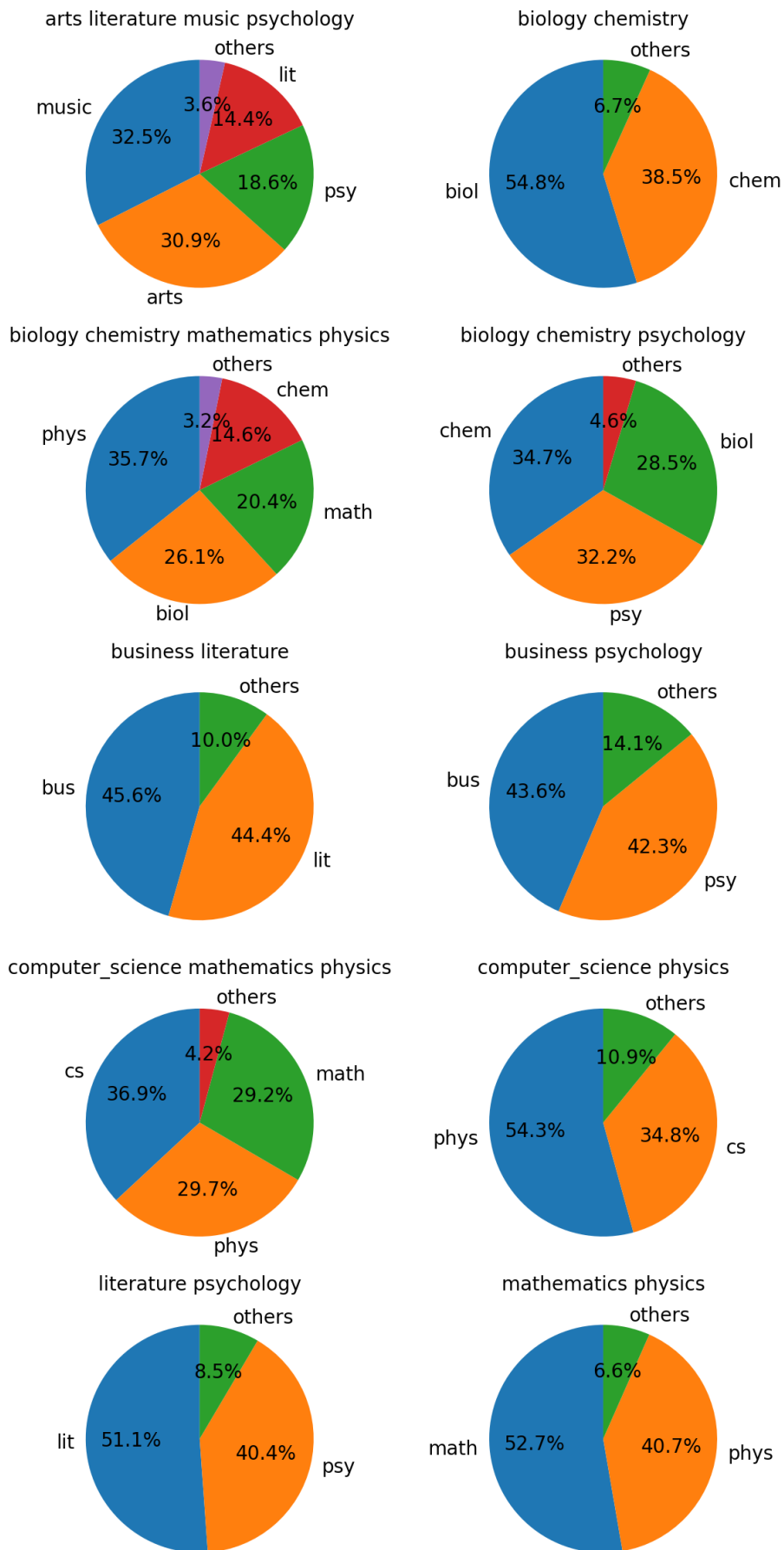


Figure 9: Watch Percentages of disciplines for each profile

4.2 Summary

To account for the large number of runs conducted and figures created, all the runs were analyzed in an identical fashion with the example run presented and the key findings were summarized in **Table 3**.

	Profiling	Recommending	Avg. View Count
Run 1	High similarity scores. Little variation. Mode: 49/94 similarities at 0.95	In 10/10 charts disciplines fully match profiles. Some degree of variation in discipline proportion. ‘others’ exceeds 10% in 6/10 charts	2129
Run 2	Very high similarity scores. Little to no variation. Mode: 73/94 similarities at 0.95	In 10/10 charts disciplines fully match profiles. Some degree of variation in discipline proportion. ‘others’ section exceeds 10% in 2/10 charts	1647
Run 3	Mostly high similarity scores. Moderate variation (outliers). Mode: 15/94 similarities at 0.9	In 6/10 charts disciplines fully match profiles. Some degree of variation in discipline proportion. ‘others’ exceeds 10% in 6/10 charts	1177
Run 4	Low similarity scores. Little variation (outliers). Mode: 45/94 similarities at 0.05	In 1/10 charts disciplines fully match profiles. Large degree of variation in discipline proportion. ‘others’ exceeds 10% in 10/10 charts. ‘arts’ present in 9/10 regardless of profile	952
Run 5	Either low or moderate to high similarity scores. Moderate variation. Mode: 26/94 similarities at 0.05	In 2/10 charts disciplines fully match profiles. Large degree of variation in discipline proportion. ‘others’ exceeds 10% in 10/10 charts. ‘arts’ present in 7/10 regardless of profile	772

Table 3: Summary of findings

Chapter 5

Discussion

5.1 Observations

To properly evaluate how the study parameter affects the algorithm, other variables had to be controlled across all runs. The bots created had the exact same profiles and interests in disciplines, and the same generated video dataset was used.

When profiling the user, the first two runs yielded high similarity scores with little to no variation as summarized in **Table 3**. As the similarity threshold (study parameter) kept increasing, the overall similarity distribution shifted towards lower values with larger variation. The average number of videos watched shows a negative correlation with the study parameter, as the requirements for a video to qualify increased. This also had an indirect effect on the capacity of the algorithm to profile the user, as it has less data to analyze.

With regards to recommending videos, across the first two runs the largest proportions of videos belonging to each discipline are fully matched with their respective profiles for all the bots attributed to said profile. Some variation is present, however, it can not be attributed to the different levels of interest, as the data in the pie charts is compiled for all the bots that fall within a certain profile.

As the study parameter increases, three notable situations occur. First, the number of pie charts showcasing completely matching disciplines with their profiles decreases. Second, the ‘others’ section continues to increase as other videos belonging to less significant disciplines are viewed. While this may seem counterintuitive at first, considering the total number of videos watched by any bot is dropping, the distribution of videos watched will more closely resemble the training set seen in **Table 4**. In this case, the bot will barely watch any other videos besides the initial 500. Finally, videos from the ‘arts’ category seem to be recommended regardless of whether the bot profile indicates such an interest. This may occur due to the low number of videos available in the training set. As such, the algorithm fails to properly penalize the arts category in the user vector, which leads to higher similarity scores between it and arts videos. One issue with this theory is that although the number of literature videos in the training set is low, the literature discipline’s prevalence is not evident. While this may be an effect of the interest level exhibited by the bots in general towards the arts and literature disciplines, further research is required to explain this behaviour.

Discipline	lit	arts	psy	cs	phy	bio	math	chem	bus	music
no. videos	41	42	46	48	50	50	52	54	58	59

Table 4: Training set video count per discipline distribution

Overall, runs 1 and 2 showcase that a moderate similarity threshold, leads to better profiling and video recommendation, which seems to contradict the idea of cosine similarity at face value. However, videos within this dataset are represented by one single discipline. The algorithm

attempts to calculate the similarity between a video vector with only one active dimension and the user vector with multiple. Thus, scores naturally fall more around the 0.3 to 0.5 values rather than very high ones. By enabling videos to be recommended to the user with these similarity scores, the users are more accurately profiled, which in turn leads to better recommendations being made. Across the two remaining runs, the best similarity threshold to permit very high accurate profiling and a solid video recommendation is 0.4 matching run 2. As it can be observed in **Fig. 7**, the similarity distribution showcases 73 out of 94 total data points to be in the 0.95 value. Additionally, the ‘others’ section in all pie charts from **Fig. 9** is kept at a minimum for the majority of available profiles.

To further evaluate the algorithm’s performance with the optimal study parameter (a similarity threshold of 0.4), we now examine its ability to distinguish between varying levels of interest. The distribution of the Spearman’s Rank Correlation coefficients was plotted through the use of *scipy.stats.spearmanr* method (SciPy Developers, 2024) in **Fig. 10**. As it can be observed, the data resembles a normal distribution around high values of coefficients, with the mode residing at 0.7 holding 37/94 data points. This indicates that the algorithm has a relatively strong ability to profile different levels of interest.

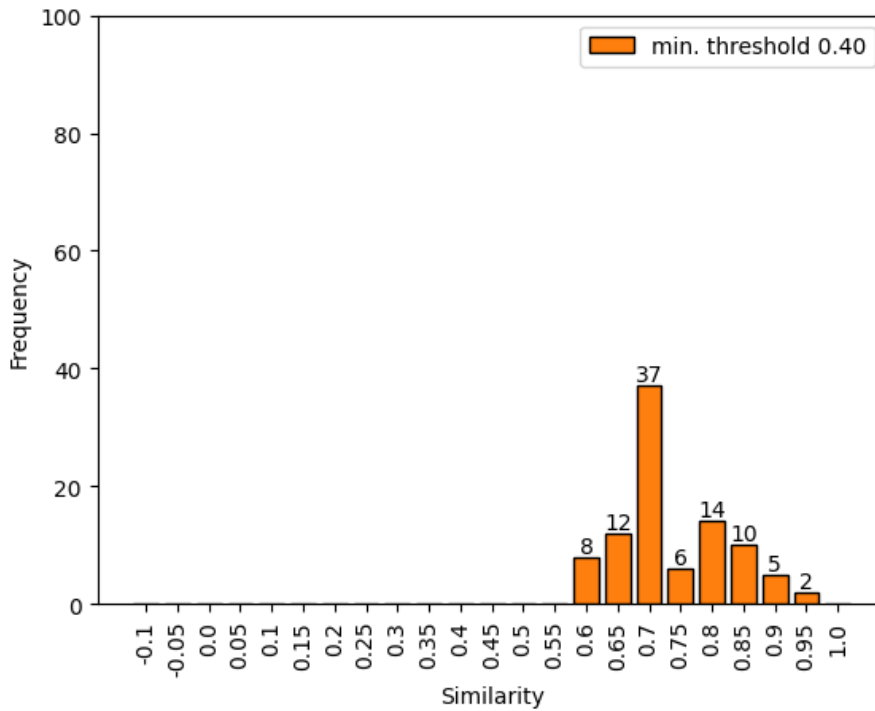


Figure 10: Bot and User Vector Spearman’s rank correlation distribution

5.2 Limitations

When building simulated environments, it is important to make them representative of an actual real life population, such that the results are statistically significant. Although an attempt was made to build the bot population based on real-life data, their actions did not accurately represent a live population. The interest level distributions were modeled on expected user reactions to content they either enjoy or do not.

Building on top of the previous limitation, this study could have benefitted from a live population utilizing the platform. While the simulated environment helped with identifying the optimal study parameter, a live population could have enabled the evaluation of the demo platform created.

Although, using cosine similarity as the sole metric to recommend videos, kept the complexity of the algorithm at a minimum, exploring other filtering methods, especially HBF is important. The CBF method (which was implemented in this thesis with cosine similarity) does have shortcomings when the items being recommended lack the necessary features (Roy & Dutta, 2022). While the dataset used did not suffer from this issue, this could happen in deployment. Thus, by implementing UBF as well, transforming the whole recommendation system into HBF, one could overcome the drawbacks of both methods.

Chapter 6

Conclusion and Further Exploration

This thesis provides a cosine similarity based video recommendation system for educational platforms. A simulated environment comprising 100 bots and 5000 pseudo-generated videos was created to simulate students utilizing this service. The environment was used to evaluate the profiling and recommending capabilities of the algorithm.

Five runs were then conducted to determine the optimal minimum similarity threshold. The results indicate that the recommendation system is able to profile users' interests, ranking them appropriately and to accurately recommend videos using the optimal threshold value of 0.4.

The implementation of the recommendation system into educational platforms can have significant implications. By accurately profiling students' interest in various disciplines, this information can be provided to individuals that have a difficult time identifying them, thus, aiding in the orientation process for further education. Moreover, further research could determine whether a recommendation system could enhance students' engagement with educational platforms, mirroring the effect observed in social media applications.

It would be important to conduct a future study on a deployed platform, which fully benefits from a live population, using the optimal study parameter found. The recommendation system could be switched to hybrid-based filtering method to account for the possible missing features in the new dataset. Furthermore, alongside studying user engagement, it would be worthwhile to investigate whether the short-form content commonly used in social media applications can enhance the educational process for individuals. This would probably generate valuable information in the field of education.

List of Figures

1	Video Retrieval pseudocode	8
2	Login Page Interface	9
3	Sign Up Page Interface	10
4	For You Page Interface	10
5	Interest Level distributions	12
6	Simulated Environment pseudocode	13
7	Bot and User Vector cosine similarities distribution	15
8	Bot view counts	15
9	Watch Percentages of disciplines for each profile	16
10	Bot and User Vector Spearman's rank correlation distribution	19

References

- Cunningham, P., & Delany, S. J. (2021). K-nearest neighbour classifiers - a tutorial. *ACM Computing Surveys*, 54(6), 1–25. <https://doi.org/10.1145/3459665>
- Data mining: Concepts and techniques*. (2012). Elsevier. <https://doi.org/10.1016/c2009-0-61819-5>
- Elastic. (2024). What is vector embedding? <https://www.elastic.co/what-is/vector-embedding>
- GeeksforGeeks. (2024). Spearman’s rank correlation. <https://www.geeksforgeeks.org/spearman-rank-correlation/>
- Khan Academy. (2024a). About khan academy. <https://www.khanacademy.org/about>
- Khan Academy. (2024b). Vector magnitude and normalization. <https://www.khanacademy.org/computing/computer-programming/programming-natural-simulations/programming-vectors/a/vector-magnitude-normalization>
- Kreutz, C. K., & Schenkel, R. (2022). Scientific paper recommendation systems: A literature review of recent publications. *International Journal on Digital Libraries*, 23(4), 335–369. <https://doi.org/10.1007/s00799-022-00339-w>
- NumPy Developers. (2024). Numpy documentation. <https://numpy.org/doc/1.26/>
- OECD. (2023). Education enrolment by field [Retrieved June 1, 2023. Note: This link has migrated; for the updated link, refer to the OECD Statistics homepage or contact the organization]. https://stats.oecd.org/Index.aspx?DataSetCode=EDU_ENRL_FIELD
- OpenStax. (2024). *Statistics*. <https://openstax.org/books/statistics/pages/2-6-skewness-and-the-mean-median-and-mode>
- Recommender systems handbook*. (2022). Springer US. <https://doi.org/10.1007/978-1-0716-2197-4>
- Rizkallah, S., Atiya, A. F., & Shaheen, S. (2021). New vector-space embeddings for recommender systems. *Applied Sciences*, 11(14), 6477. <https://doi.org/10.3390/app11146477>
- Roy, D., & Dutta, M. (2022). A systematic review and research perspective on recommender systems. *Journal of Big Data*, 9(1). <https://doi.org/10.1186/s40537-022-00592-5>
- SciPy Developers. (2024). Scipy documentation. <https://docs.scipy.org/doc/scipy/>
- Singh, R. H., Maurya, S., Tripathi, T., Narula, T., & Srivastav, G. (2020). Movie recommendation system using cosine similarity and knn. *International Journal of Engineering and Advanced Technology*, 9(5), 556–559. <https://doi.org/10.35940/ijeat.e9666.069520>
- Wang, P. (2022). Recommendation algorithm in tiktok: Strengths, dilemmas, and possible directions. *International Journal of Social Science Studies*, 10(5), 60. <https://doi.org/10.11114/ijsss.v10i5.5664>